SPANS-99: Best Practice Summary

> **Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/25/2026

Definitive best practice settings for distributed tracing and span analysis. Each entry specifies the exact configuration.

Table of Contents

1. DQL Syntax Rules
2. Filtering & Performance
3. Time Range & Cost
4. Duration & Aggregation
5. Trace Analysis
6. Service Dependencies
7. Security
8. OpenPipeline Span Configuration
9. Bucket Strategy & Sampling

---

1. DQL Syntax Rules

Practice	Recommended Setting/Value	Priority
Equality operator	`==` (double equals) — never `=`	Critical
Array literals	`{"a", "b"}` with curly braces and double quotes — never `('a', 'b')`	Critical

Practice	Recommended Setting/Value	Priority
NULL checks	`isNull(field)` / `isNotNull(field)` — never `== null`	Critical
Multi-value matching	`in(field, {"val1", "val2"})` — never SQL `IN`	Critical
`span.kind` values	Lowercase only: `"server"`, `"client"`, `"internal"`, `"producer"`, `"consumer"`	Critical
`span.status_code` values	Lowercase only: `"error"`, `"ok"`, `"unset"`	Critical
String literals	Double quotes everywhere — never single quotes	Critical
Grouping syntax	`summarize count(), by:{field}` with curly braces — never `GROUP BY`	Critical
Always alias aggregation results	`summarize error_count = count()` — required for `sort` and `fieldsAdd`	Critical

2. Filtering & Performance

Practice	Recommended Setting/Value	Priority
Filter immediately after fetch	All `filter` commands before any `fieldsAdd`, `summarize`, `sort`	Critical
Filter on indexed fields first	`trace.id`, `span.id`, `dt.entity.service`, `span.name`, `span.kind`, `span.status_code`, `start_time`	Critical
Use `dt.entity.service` over `service.name`	Always populated and indexed vs. sometimes missing and not indexed	Recommended
`==` for exact matches, `~` for wildcards only	`field == "exact"` is faster than `field ~ "exact"`	Recommended
Combine conditions in single filter	`filter span.kind == "server" and isNotNull(dt.entity.service) and duration > 100ms`	Recommended

Practice	Recommended Setting/Value	Priority
Most restrictive filters first	Order from most selective to least selective	Recommended
Select only needed fields	`fields start_time, dt.entity.service, span.name, duration` after filtering	Recommended
Compute derived fields after filtering	`fieldsAdd duration_ms = ...` must appear after all `filter` steps	Recommended
Sort after summarize, never after fetch	Sorting before aggregation wastes resources	Critical
Limit after sort, at pipeline end	`sort desc | limit 20` as final commands — never `limit` before `summarize`	Critical

3. Time Range & Cost

Practice	Recommended Setting/Value	Priority
Always specify explicit time range	`fetch spans, from:-1h` — never bare `fetch spans`	Critical
Use narrowest range possible	Debugging: `from:-5m`. Recent: `from:-1h`. Daily: `from:-24h`. Weekly: `from:-7d`	Critical
Target specific buckets	`fetch spans, bucket:{"default_spans"}` — avoid scanning all buckets	Recommended
Never group by `trace.id` without pre-filtering	Millions of unique values — filter by error or time first	Critical
Group by low-cardinality fields for dashboards	`dt.entity.service`, `span.name`, `span.kind`, `http.route` — never `url.path`, `trace.id`	Recommended
Always `limit` non-aggregated queries	`limit 100` on any raw span query	Critical

Practice	Recommended Setting/Value	Priority
Use only needed percentiles	p50 + p95 for monitoring; add p99 only for SLO tracking	Recommended

4. Duration & Aggregation

Practice	Recommended Setting/Value	Priority
Convert duration from nanoseconds	`/1000000.0` for ms, `/1000000000.0` for seconds	Critical
Use duration literals in filters	`filter duration > 100ms`, `filter duration > 1s`	Recommended
Use `countIf()` for conditional counts	`summarize errors = countIf(span.status_code == "error"), total = count()` in single pass	Recommended
Error rate calculation	`(error_count * 100.0) / total_requests` — use `100.0` (float) to avoid integer division	Recommended
Percentile trends with `bin()`	`fieldsAdd time_bucket = bin(start_time, 10m) | summarize p95 = percentile(duration, 95)/1000000, by: {time_bucket}` — `makeTimeseries` does NOT support `percentile()`	Critical
Use `makeTimeseries` for dashboard charts	`makeTimeseries requests = count(), errors = countIf(span.status_code == "error"), interval:5m`	Recommended
Minimum sample size before ranking	`filter request_count > 10` before percentile/error rate rankings	Recommended

5. Trace Analysis

Practice	Recommended Setting/Value	Priority
Find root spans	`filter isNull(span.parent_id)`	Recommended

Practice	Recommended Setting/Value	Priority
Reconstruct trace chronologically	`filter trace.id == "ID" | sort start_time asc`	Recommended
Find first error in a trace	`summarize first_error_time = min(start_time), service = takeFirst(service.name), by:{trace.id}` after filtering errors	Recommended
Detect cascading failures	`filter span.status_code == "error" | summarize count(), by:{trace.id} | filter count > 1`	Recommended
Impact score for bottleneck spans	`fieldsAdd impact_score = call_count * avg_duration_ms | sort impact_score desc`	Recommended
Health scorecard classification	`if(error_rate > 5, "Critical", else: if(error_rate > 1, "Warning", else: "Healthy"))`	Recommended

6. Service Dependencies

Practice	Recommended Setting/Value	Priority
Map dependencies via CLIENT spans	`filter span.kind == "client" | summarize count(), by:{service.name, server.address}`	Recommended
Use `peer.service` for named dependencies	`filter span.kind == "client" and isNotNull(peer.service)` when available	Recommended
Track async messaging	`filter span.kind in {"producer", "consumer"} | summarize count(), by: {messaging.system, messaging.destination.name}`	Recommended
Outbound/inbound ratio	`summarize inbound = countIf(span.kind == "server"), outbound = countIf(span.kind == "client"), by: {service.name}` — high ratio = heavy downstream dependency	Optional

7. Security

Practice	Recommended Setting/Value	Priority
Use `http.route` in dashboards, never `url.path`	`http.route` = pattern (no PII), `url.path` = actual values (may contain PII)	Critical
Weekly PII audit: emails in URLs	`filter contains(url.path, "@") or contains(url.path, "%40")` — must return 0	Critical
Weekly audit: tokens in URLs	`filter contains(url.path, "password") or contains(url.path, "token") or contains(url.path, "secret")`	Critical
Weekly audit: sensitive data in `db.statement`	`filter contains(db.statement, "password") or contains(db.statement, "ssn")`	Critical
Monitor 401/403/429 as security signals	`filter in(http.response.status_code, {401, 403, 429})` with 10-min bucketing on dashboard	Critical
Brute force detection	401 count >10 per hour per endpoint = alert	Critical
Enumeration detection	404 count >100 with high unique path count = scanning alert	Recommended
Limit `trace.id` exposure	`takeFirst(trace.id)` per error pattern — never expose all trace IDs	Recommended
Mask emails in span URLs via OpenPipeline	Replace `[a-zA-Z0-9._%+-]+@...` with `***@***.***` on `http.url`	Critical
Mask tokens in query params via OpenPipeline	Replace `(token|key|secret|password)=([^&]+)` with `$1=***REDACTED***`	Critical
Monthly full PII audit	Check: emails, tokens, user IDs in URLs + sensitive data in `db.statement` + `exception.stacktrace`	Critical

8. OpenPipeline Span Configuration

Practice	Recommended Setting/Value	Priority
Drop health check spans at ingestion	`contains(span.name, "health") or contains(span.name, "ready")`	Critical
Drop static asset spans	`endsWith(span.name, ".js") or endsWith(span.name, ".css") or endsWith(span.name, ".png")`	Recommended
Never drop error spans	`span.status_code == "error"` → keep at 100% regardless of sampling	Critical
Never drop slow spans	`duration > 1s` → keep at 100% regardless of sampling	Critical
Pre-compute `duration_ms` at ingestion	Transform: `duration_ms = duration / 1000000`	Optional
Route to environment buckets	Production: `spans_production_90d`. Staging: `spans_staging_7d`. Dev: `spans_default_3d`	Recommended
Route sensitive service spans to restricted buckets	Payment/auth services → `spans_sensitive` with restricted IAM	Recommended
Bucket-level IAM for team isolation	Frontend team: `spans_frontend` only. SRE: all span buckets. Compliance: `audit_spans`	Recommended

9. Bucket Strategy & Sampling

Practice	Recommended Setting/Value	Priority
Retention by environment	Production: 90 days. Standard: 35 days. Staging: 7 days. Dev: 3 days	Recommended
Three-tier sampling	(1) Errors → 100%. (2) Slow >1s → 100%. (3) Everything else → 10-50%	Critical

Practice	Recommended Setting/Value	Priority
Sample high-volume services at 10%	Services generating >10,000 spans/hour	Recommended
Calculate savings before filtering	Count health checks + static + errors to quantify droppable vs must-keep	Recommended
Extract `dt.ingest.size` as metric	Key: `span.ingest.size.by.app`, value: `dt.ingest.size`, dim: app identifier	Recommended
Verify bucket distribution monthly	`summarize count(), by: {dt.system.bucket}` — verify routing rules working	Recommended

---